DESIGN AND ANALYSIS OF ALGORITHMS

Project Report

Lightpath: Latency-Aware Cloud Task Placement on Network Graphs

Graph Algorithms, Dynamic Programming and Greedy Scheduling for Replica and VM Placement



Team Members

Naman Singla – 25WU0101084

Mohak Gupta – 25WU0101074

Faculty

Dr. Monday Jubrin Abdullahi

Section : CSE RHINOS

Academic Year : 2026–2027

Lightpath: Latency-Aware Cloud Task Placement on Network Graphs

Naman Singla (25WU0101084) Mohak Gupta (25WU0101074)
Guided by Dr. Monday Jubrin Abdullahi • Section CSE RHINOS • Woxsen University

Abstract—Cloud services run as replicated virtual machines (VMs) spread over a network of servers. Where those replicas sit decides how long every user request waits. We model the infrastructure as a weighted graph whose nodes are sites and whose edge weights are link latencies, and we place service replicas to minimise demand-weighted mean latency subject to CPU, memory and budget constraints. The problem contains the p-median problem and is NP-hard, so we combine exact polynomial algorithms where they apply with greedy heuristics where they do not: minimum spanning trees (Prim, Kruskal) for the provisioning backbone, single- and all-pairs shortest paths (Dijkstra, Bellman-Ford, Floyd–Warshall, Johnson) for routing and the latency matrix, 0/1 and multiple-choice knapsack dynamic programs for budgeted selection, four greedy schedulers refined by local search, and a mixed-integer program that certifies optimality on small instances. We evaluate on ten networks, including the real Abilene and NSFNET backbones, 25 Indian and 24 Nigerian cities at real coordinates, and the Woxsen University campus. Greedy placement with local search lands within 0.5 % of the proven optimum on average (worst case about 3 %) at roughly a fifth of the exact solver's time. Under tight capacity, packing-first placement remains feasible where latency-first greedies fail. All results are reproducible through a tested Python package, a REST API and an interactive web dashboard.

Keywords— replica placement, p-median, shortest paths, minimum spanning tree, knapsack, submodularity, local search, mixed-integer programming.

1 Introduction

A request to an online service travels from the user's nearest network site to a server that holds a copy (*replica*) of the service. Its delay is dominated by propagation along fibre (about 5 μs per km) and per-hop switching. Placing replicas close to demand therefore reduces latency, but servers have finite CPU and memory, replicas cost money, and a server chosen for one service may be needed by another. Deciding where to run what is a combinatorial optimisation problem on a graph.

This project builds *Lightpath*, a complete placement system, and uses it to study three questions:

Q1 Which classical graph algorithms are appropriate at each stage, and how do their theoretical bounds compare with measured behaviour in Python?

Q2 How close do fast greedy schedulers get to the true optimum, and what does closing the gap cost?

Q3 How do network size, density, capacity pressure and budget change the answer?

Contributions. (i) A clean formulation of capacitated, budgeted, multi-service replica placement and a 7-stage pipeline that solves it (Section 2, 4). (ii) Implementations of every algorithm from first principles, including binary, Fibonacci and pairing heaps and union-find, with correctness arguments and complexity bounds. (iii) A transfer-cost model in which some links have negative cost but negative cycles provably cannot occur, which motivates Bellman-Ford and Johnson. (iv) Eight experiments with an exact MILP reference. (v) Policy outputs (routes, schedules, cost/benefit tables) and an interactive dashboard for India, Nigeria and the Woxsen campus.

2 Problem Formulation

Network. $G = (V, E)$ is an undirected graph of $n = |V|$ sites and $m = |E|$ links. Link (u, v) has latency ℓ_{uv} (ms) and bandwidth b_{uv} (Gbit/s). For geographic networks we use

$$\ell_{uv} = 1.3 \cdot \frac{d_{\mathrm{km}}(u, v)}{200 \text{ km/ms}} + 0.3 \text{ ms}, \tag{1}$$

where 200 km/ms is the speed of light in fibre, 1.3 accounts for cable detours, and 0.3 ms models switching. $d(u, v)$ denotes the shortest-path latency between sites.

Servers and services. A subset $S \subseteq V$ are servers with CPU capacity C_s, memory M_s and activation cost f_s; routers have $C_s = 0$. Each service $j \in J$ needs c_j vCPU and μ_j GB per replica, costs p_j per replica-hour, has priority π_j, demand r_{jc} (requests/s) at client sites c, and between $k_j^{\min}$ and $k_j^{\max}$ replicas.

Decision and objective. Binary $y_{sj} = 1$ if a replica of j runs on s (at most one per server, for fault isolation). Each client is served by its nearest replica, so with weights $w_{jc} = \pi_j r_{jc}$ we minimise

$$L(y) = \frac{\sum_j \sum_c w_{jc} \min_{s : y_{sj} = 1} d(c, s)}{\sum_j \sum_c w_{jc}} \tag{2}$$

subject to $\sum_s y_{sj} = k_j$, $k_j^{\min} \le k_j \le k_j^{\max}$, $\sum_j c_j y_{sj} \le C_s$, $\sum_j \mu_j y_{sj} \le M_s$ for every server, and the hourly budget $\sum_j p_j k_j \le B$. An optional server budget $\sum_{s \text{ active}} f_s \le B_S$ switches servers off.

Hardness. With one service, unlimited capacity and fixed k, the problem is exactly the p-median problem, which is NP-hard on general graphs [1]. Capacities add bin-packing structure, which is NP-hard by itself. This justifies exact algorithms for the polynomial sub-problems (paths, trees, pseudo-polynomial knapsacks) and heuristics with a measured optimality gap for placement.

Transfer costs with negative edges. Replicas of the same service synchronise data. Moving data $u \to v$ costs

$$\kappa(u \to v) = \beta + \gamma \ell_{uv} + \varphi(u) - \varphi(v) \quad (\$/\mathrm{GB}), \tag{3}$$

where φ is a regional price level (lower at core data centres, with a deterministic per-site jitter). Arcs into higher-priced regions become negative: a rebate.

Lemma 1 *The directed graph with weights* (3) *has negative arcs but no negative cycle.*

Proof sketch. Around any directed cycle the terms $\varphi(u) - \varphi(v)$ telescope to zero, so the cycle's cost is $\sum(\beta + \gamma \ell_{uv}) > 0$. □

Lemma 1 is the same potential argument that underlies Johnson's reweighting. It guarantees that Bellman-Ford and Johnson always succeed, while Dijkstra is unsafe because of the negative arcs. In the India network 24 of 88 directed arcs are negative.

3 Datasets

Table 1 lists the ten instances. Abilene (Internet2) and NSFNET T1 are real research backbones. India (24 metros plus Woxsen University at Kamkole) and Nigeria (24 cities, with Lagos, Abuja and Port Harcourt as core data centres) use real coordinates with links synthesised to each city's three nearest neighbours; they are not carrier maps. The Woxsen campus has 23 sites named and arranged from the official campus map (Main Entrance in the west to Student Hostels A–D in the east), with sub-millisecond links, two data centres (Academic Block and Admin Block) and campus workloads such as the learning portal, Bloomberg feed, AI-lab GPU jobs and sports live streaming. Synthetic families cover geographic WANs (Waxman [2]), random graphs (Erdős–Rényi [3]), scale-free ISP-like graphs (Barabási–Albert [4]) and a $k = 4$ data-centre fat-tree [5]. Server tiers (core 48 vCPU, regional 24, edge 8, routers 0) are assigned by degree centrality. Services draw Zipf-like demand clustered around a random centre.

Table 1: Datasets. Load is the CPU needed by minimum replicas divided by total capacity; best is the recommended plan's mean latency.

Name	Source	Sites	Links	Servers	Services	Load	Best (ms)
india	real coordinates + Woxsen	25	44	22	8	18%	2.326
nigeria	real coordinates	24	47	21	8	23%	1.137
woxsen	campus map	23	29	13	8	17%	0.055
abilene	real topology	11	14	10	6	42%	3.827
nsfnet	real topology	14	21	12	7	26%	5.220
fat-tree-k4	data centre	36	48	16	6	18%	0.022
waxman-60	synthetic WAN	60	89	51	14	13%	5.245
waxman-60-tight	0.5× capacity	60	89	51	14	27%	6.686
er-100	Erdős–Rényi	100	350	85	16	10%	5.062
ba-150	Barabási–Albert	150	296	128	20	7%	7.594

4 Algorithms, Correctness and Complexity

Fig. 1 shows the pipeline; Table 2 summarises the bounds.

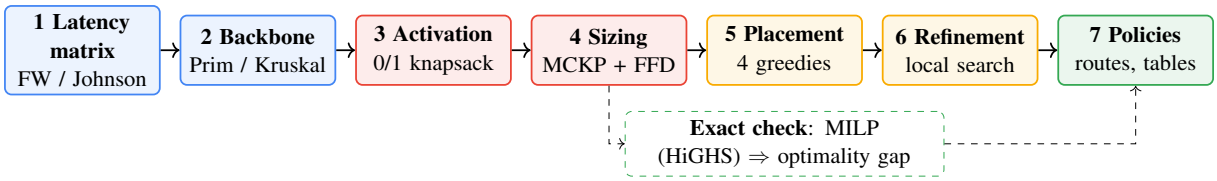


Figure 1: The Lightpath pipeline. Stages 1–2 are exact polynomial algorithms, 3–4 pseudo-polynomial dynamic programs, 5–6 heuristics whose gap is measured by the exact MILP.

4.1 Data structures

The graph is stored as adjacency lists (one hash map per vertex), giving $\Theta(n+m)$ space and $O(\deg u)$ neighbour scans. Four addressable priority queues share one interface (*push*, *pop*, *decrease-key*): a binary heap with a position map ($O(\log n)$ for all operations), a Fibonacci heap with cascading cuts (amortised $O(1)$ insert and decrease-key, $O(\log n)$ extract-min [6]), a two-pass pairing heap, and Python's `heapq` with lazy deletion. Kruskal uses union-find with union by rank and path compression, $O(\alpha(n))$ amortised per operation [7].

4.2 Minimum spanning tree (backbone)

The MST is the cheapest set of links that keeps every site connected, which is the first set of links to provision. Prim grows one tree from a root and repeatedly adds the lightest edge leaving it [8]. Kruskal scans edges in increasing weight and keeps those that join two components [9]. *Correctness* for both follows from the *cut property*: for any cut $(X, V \setminus X)$ a lightest crossing edge belongs to some MST. Prim's choice is such an edge for the cut (tree, rest), and Kruskal's for the cut separating either endpoint's component. Disconnected inputs yield a spanning forest. *Complexity:* Prim $O((n+m)\log n)$ with a binary heap and $O(m+n\log n)$ with a Fibonacci heap; Kruskal $O(m\log m)$ for sorting plus $O(m\,\alpha(n))$. The pipeline selects Prim when $m > n\log_2 n$ and Kruskal otherwise.

4.3 Single-source shortest paths

Dijkstra (Algorithm 1) [10]. *Invariant:* when u is extracted, $dist[u]$ equals the true distance $\delta(s,u)$. Suppose u is the first vertex extracted with $dist[u] > \delta(s,u)$. On a shortest $s \rightsquigarrow u$ path, let y be the first unsettled vertex and x its settled predecessor. Then $dist[y] = \delta(s,y) \le \delta(s,u) < dist[u]$ because weights are non-negative, so y would have been extracted first, a contradiction. Time: $O((n+m)\log n)$ with a binary heap, $O(m+n\log n)$ with a Fibonacci heap. Our implementation rejects negative weights explicitly.

Algorithm 1 Dijkstra with an addressable heap

Input: graph G, non-negative weights w, source s

```
dist[v] ← ∞ for all v;  dist[s] ← 0;  Q ← {s}
while Q ≠ ∅ do
    u ← ExtractMin(Q)                                   ▷ u is settled: dist[u] = δ(s,u)
    for each edge (u,v) with v not settled do
        if dist[u] + w(u,v) < dist[v] then
            dist[v] ← dist[u] + w(u,v);  parent[v] ← u;  PushOrDecrease(Q,v)
```

Bellman-Ford [11, 12] relaxes every arc in rounds. By induction, after round i every vertex whose shortest path uses at most i arcs has its correct distance; simple paths have at most $n-1$ arcs, so $n-1$ rounds suffice, and any improvement in round n certifies a reachable negative cycle (which we extract by walking parent pointers). Time $O(nm)$, with early exit when a round changes nothing. It computes cheapest replica-synchronisation routes under (3).

4.4 All-pairs shortest paths

Floyd–Warshall [13, 14] is dynamic programming over intermediate vertices: with $d_k(i,j)$ the shortest $i \rightsquigarrow j$ distance using intermediates in $\{1..k\}$, $d_k(i,j) = \min\{d_{k-1}(i,j),\ d_{k-1}(i,k) + d_{k-1}(k,j)\}$, because a shortest path either avoids k or passes through it once (no negative cycles). $\Theta(n^3)$ time, $\Theta(n^2)$ space, and a negative diagonal entry exposes a negative cycle. We provide a pure-Python version and a NumPy version that vectorises the (i,j) loop; both are still $\Theta(n^3)$. **Johnson** [15] adds a vertex q with zero-cost arcs to all vertices, computes potentials $h = \text{Bellman-Ford}(q)$, and reweights $w'(u,v) = w(u,v) + h(u) - h(v) \geq 0$ (triangle inequality). Every $s \rightsquigarrow t$ path changes by the constant $h(s) - h(t)$, so shortest paths are preserved and Dijkstra can run from every vertex: $O(nm + n(n+m)\log n)$. **Choice rule.** Counting both directions of each undirected link, the pipeline treats a graph as sparse when $m \log_2 n < n^2$. Sparse graphs use repeated Dijkstra (or Johnson if any weight is negative); dense graphs use vectorised Floyd–Warshall. Section 7 tests this rule.

4.5 Knapsack dynamic programs (budgeted selection)

0/1 knapsack (server activation). Each server has value b_s, the capacity-scaled latency saving it offers relative to an average server, and weight f_s. With costs scaled to integer cents (W = budget in cents), $\text{OPT}[i][w] = \max\{\text{OPT}[i-1][w],\ \text{OPT}[i-1][w-f_i] + b_i\}$. *Correctness:* by exchange, an optimal set for items $1..i$ either excludes item i (optimal for $1..i-1$ at the same budget) or includes it (item i plus an optimal set at budget $w - f_i$). Time and space $\Theta(nW)$, which is pseudo-polynomial; one bit per cell is kept for reconstruction. Because b_s is an additive surrogate, a repair step re-adds high capacity-per-dollar servers if the minimum replicas would not fit.

Multiple-choice knapsack (replica sizing). For each service the options are $k \in [k_j^{\min}, k_j^{\max}]$ with cost kp_j and value equal to the weighted-latency saving over $k_j^{\min}$, estimated by an uncapacitated greedy placement. Exactly one option per group gives $\text{OPT}[g][w] = \max_{o \in g} \text{OPT}[g-1][w-w_o] + v_o$ in $\Theta(W \sum_g |g|)$. Since this estimate ignores bin packing, a *capacity repair* then removes the largest extra replicas until a first-fit-decreasing (FFD) packing exists. That guarantees at least one feasible placement downstream.

4.6 Greedy schedulers and local search

Let $F_j(T) = \sum_c w_{jc}\,(P - \min_{s \in T} d(c,s))$ be the latency *saving* of placing service j on server set T (with P a large constant). F_j is monotone and submodular: adding a server helps less when more servers are already open.

- **Demand-first:** services in decreasing demand; each replica goes to the server nearest the next demand

hotspot that still has room.

- **Marginal gain (CELF):** a global heap of (service, server) pairs keyed by marginal saving; stale keys are re-evaluated lazily, which is valid because submodularity means gains never increase [17]. Without capacities, greedy maximisation of a monotone submodular function under a cardinality constraint achieves at least $(1-1/e)$ of the optimal saving [16].
- **Regret:** repeatedly place the service whose best and second-best feasible servers differ most, the classic generalised-assignment heuristic [18]. It protects services with few good options from being crowded out.
- **FFD packing:** replicas sorted by CPU descending, each placed on a fitting server chosen by latency saving among the tightest-fitting half. If that gets stuck it falls back to plain FFD, which uses at most $\frac{11}{9}\mathrm{OPT}+\frac{6}{9}$ bins for one-dimensional packing [19].

Local search repeatedly applies the best improving *relocate* (move one replica to a free server) or *swap* (exchange replicas of two services between two servers when both fit). Candidate relocations are evaluated in a vectorised way using each client's nearest and second-nearest replica. Every accepted move strictly lowers (2) and the number of placements is finite, so the search terminates in a local optimum. For uncapacitated k-median, single-swap local optima are within a factor 5 of optimal [20].

4.7 Exact MILP and batch scheduling

For the optimality reference we solve $\min \sum w_{jc} d(c,s) x_{csj} / \sum w_{jc}$ s.t. $\sum_s x_{csj} = 1$, $x_{csj} \le y_{sj}$, $\sum_s y_{sj} = k_j$, and the capacity rows, with y binary and $x \in [0,1]$, using HiGHS [21] through SciPy [22]. The disaggregated constraints $x \le y$ give a tight LP relaxation, which we also report as a lower bound when the MILP is too large. Services that no server can host keep the same penalty in both the MILP and the heuristics, so the objectives match. For background *batch jobs*, a list scheduler orders jobs longest-first (LPT) and assigns each to the server lane with the earliest finish time after its input data arrives (latency plus size over bottleneck bandwidth). Without transfer delays, LPT is within $\frac{4}{3}-\frac{1}{3m}$ of the optimal makespan [23].

Table 2: Algorithms used, with guarantees and complexity.

Stage	Algorithm	Guarantee / complexity
Backbone	Prim (binary / Fibonacci heap), Kruskal	exact (cut property); $O(m \log n)$ / $O(m + n \log n)$ / $O(m \log m)$
Routing	Dijkstra; Bellman-Ford	exact for $w \ge 0$; exact with negative arcs, $O(nm)$
Latency matrix	Floyd–Warshall; Johnson	exact, $\Theta(n^3)$; exact, $O(nm + n(n+m)\log n)$
Activation, sizing	0/1 knapsack; multiple-choice knapsack	exact for the surrogate values, $\Theta(nW)$; $\Theta(W \sum \lvert g \rvert)$
Placement	demand-first, CELF, regret, FFD	heuristic; $(1-1/e)$ without capacities (CELF)
Refinement	relocate + swap local search	monotone descent; factor 5 for uncapacitated k-median
Reference	MILP (HiGHS), LP relaxation	optimal / lower bound; exponential worst case
Batch	LPT earliest-finish-time	$\frac{4}{3}-\frac{1}{3m}$ without transfers; $O(T \lvert S \rvert \log L)$

5 System Design (UML)

This section describes Lightpath in standard UML views: who uses the system (use cases), how it is split into deployable parts (components), the main classes and their relationships (class diagram), what happens during one solve request (sequence diagram), and the decision flow of the solver (activity diagram).

5.1 Use-case diagram

Fig. 2 shows the three actors. A *network planner* uses the studio to load or generate networks and obtain placement policies. A *student or instructor* uses the algorithm lab and experiment pages to study the

algorithms. A *researcher* runs the experiment suite and exports data. All actions go through the dashboard or the Python API.

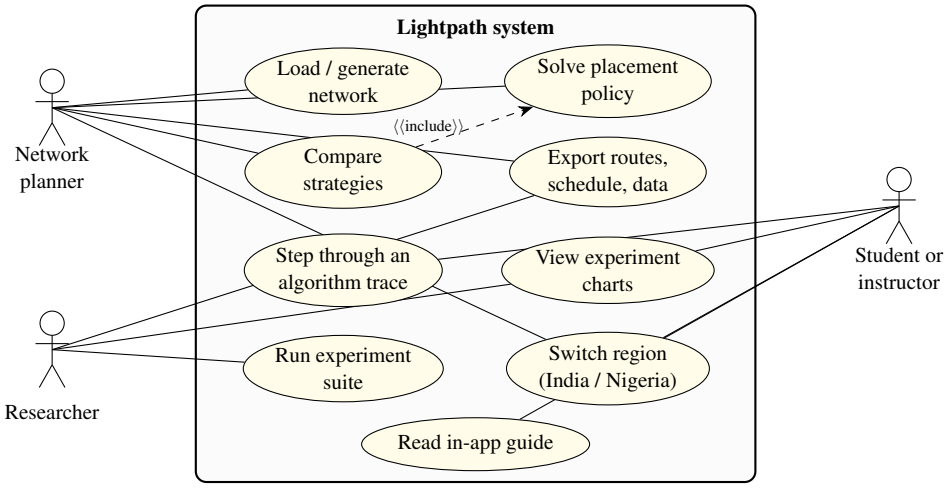


Figure 2: Use-case diagram: three actors and the nine main use cases.

5.2 Component and deployment view

Lightpath has four layers (Fig. 3). The React dashboard runs in the browser. It talks over HTTP/JSON to a FastAPI server, which also serves the built web files. The server calls the `cloudplace` solver, which uses the algorithm and core packages. When no server is running, the dashboard reads pre-computed snapshot files instead, so the same build also works offline.

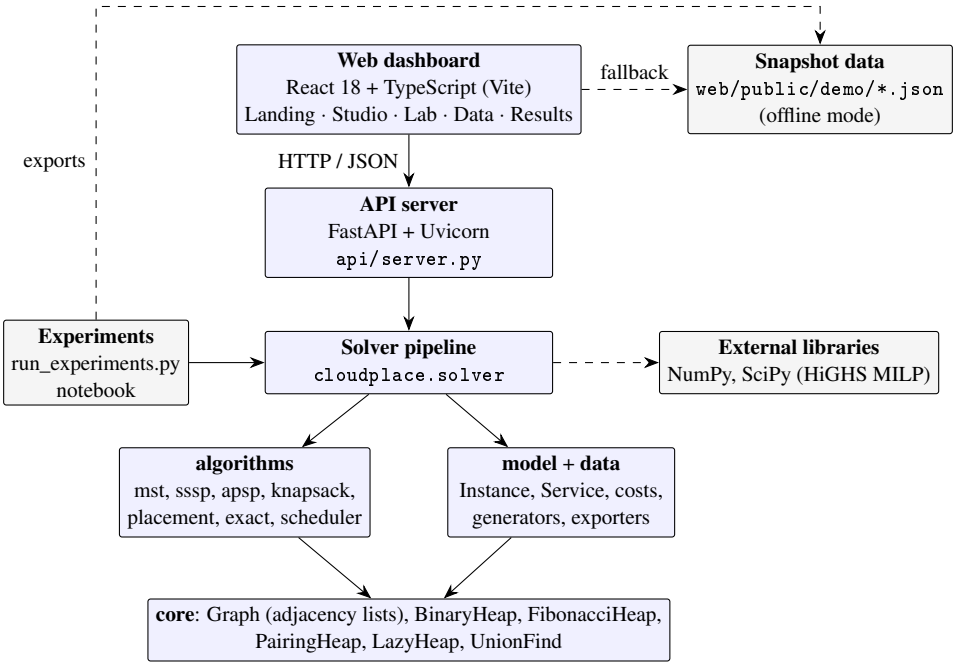


Figure 3: Component diagram. Solid arrows are calls; dashed arrows are optional or data dependencies.

5.3 Class diagram

Fig. 4 shows the main classes. An `Instance` is composed of one `Graph` and a list of `Service` and `BatchTask` objects. A `PlacementContext` wraps an instance together with its all-pairs distance matrix and provides the objective and capacity checks used by every placement algorithm. All four heaps share one interface (`push`, `pop`, `decrease_key`), so Dijkstra and Prim can be run with any of them.

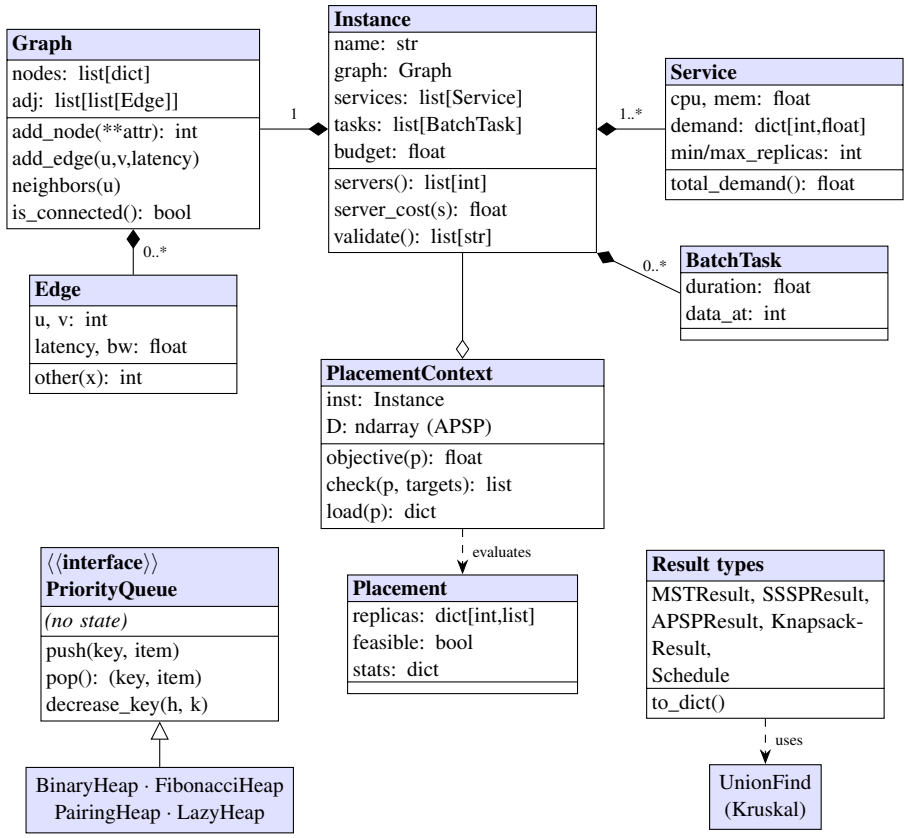


Figure 4: UML class diagram of the core model (filled diamond: composition; hollow diamond: aggregation; hollow triangle: realisation of an interface; dashed: dependency).

5.4 Sequence diagram for one solve request

Fig. 5 follows a request from the studio. The API builds an `Instance`; the solver computes the MST backbone and the all-pairs matrix, sizes replicas with the knapsack DP, runs every greedy (each followed by local search), runs the MILP when the instance is small enough, and assembles routes, schedules and recommendations into one JSON response.

5.5 Activity diagram of the solver

Fig. 6 shows the decisions inside the solver. Two of them are made automatically from graph size: Prim or Kruskal for the backbone, and a dense or sparse all-pairs method. If the plan does not fit in capacity, the solver falls back to first-fit-decreasing packing before local search.

5.6 Module and interface summary

Table 3 lists the packages and Table 4 the REST interface used by the dashboard.

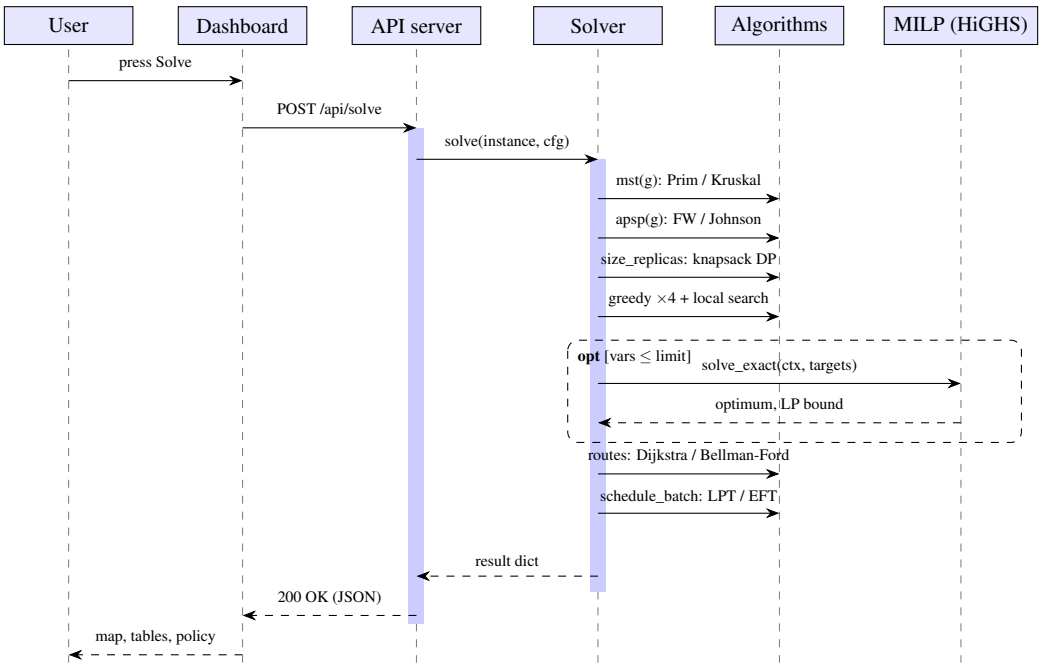


Figure 5: Sequence diagram of `POST /api/solve`. Dashed arrows are returns; the **opt** frame runs only when the MILP is small enough.

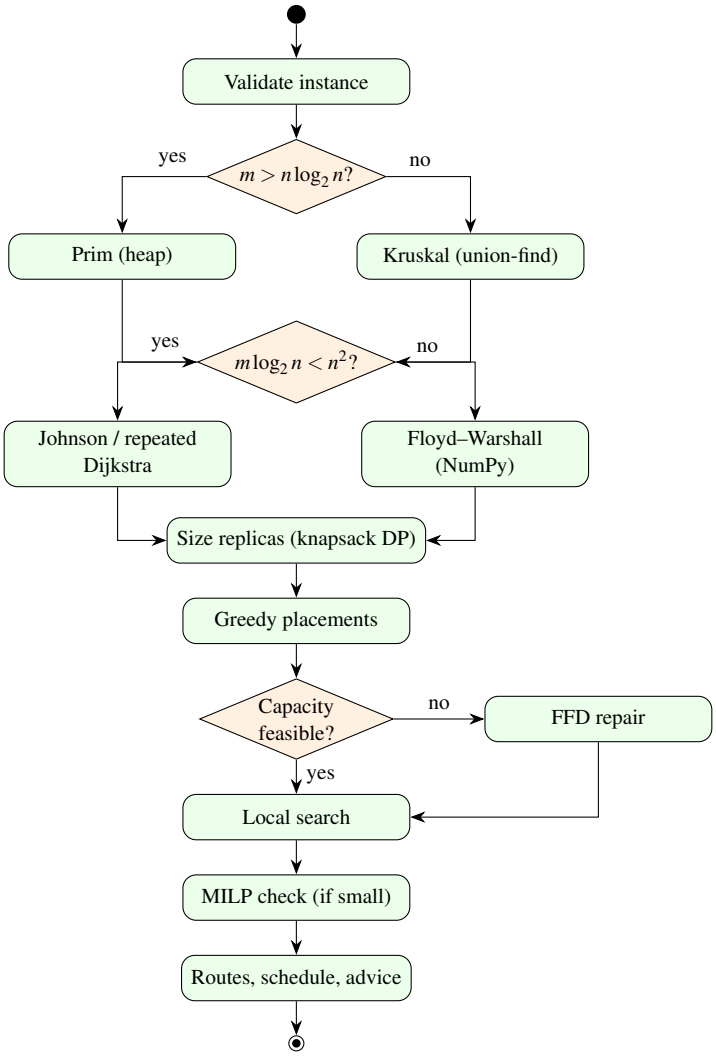


Figure 6: Activity diagram of the solver pipeline with its automatic algorithm choices.

Table 3: Package structure of `cloudplace`.

Package / module	Main contents	Responsibility
`core.graph`	Graph, Edge	Adjacency-list graph with node and edge attributes
`core.heaps`	Binary, Fibonacci, Pairing, Lazy	Priority queues with a common interface
`core.unionfind`	UnionFind	Union by rank with path compression
`algorithms.mst`	prim, kruskal, mst	Backbone tree with automatic choice
`algorithms.sssp`	dijkstra, bellman_ford	Routing; negative arcs and cycle detection
`algorithms.apsp`	floyd_warshall, johnson, apsp	All-pairs latency matrix
`algorithms.knapsack`	knapsack_01, MCKP	Server activation and replica sizing
`algorithms.placement`	4 greedies, local_search	Heuristic placement under capacity
`algorithms.exact`	solve_exact	MILP optimum and LP lower bound
`algorithms.scheduler`	schedule_batch	LPT/EFT batch task schedule
`model`	Instance, Service, costs	Problem model and transfer-cost graph
`data`	generators, export	10 datasets; JSON, GEXF, CSV export
`solver`	solve, SolveConfig	End-to-end pipeline and policy output

Table 4: REST endpoints exposed by `api/server.py`.

Method	Path	Purpose
GET	`/api/health`	Liveness check used by the dashboard to pick live or offline mode
GET	`/api/datasets`	List the built-in networks
GET	`/api/datasets/{name}`	Return one network with services and tasks
POST	`/api/generate`	Generate a synthetic network (Waxman, ER, BA, fat-tree)
POST	`/api/solve`	Run the full pipeline and return the placement policy
POST	`/api/algorithms/run`	Run one algorithm and return its step-by-step trace
GET	`/api/experiments`	Return experiment results for the charts
GET	`/{path}`	Serve the built dashboard (guarded against path traversal)

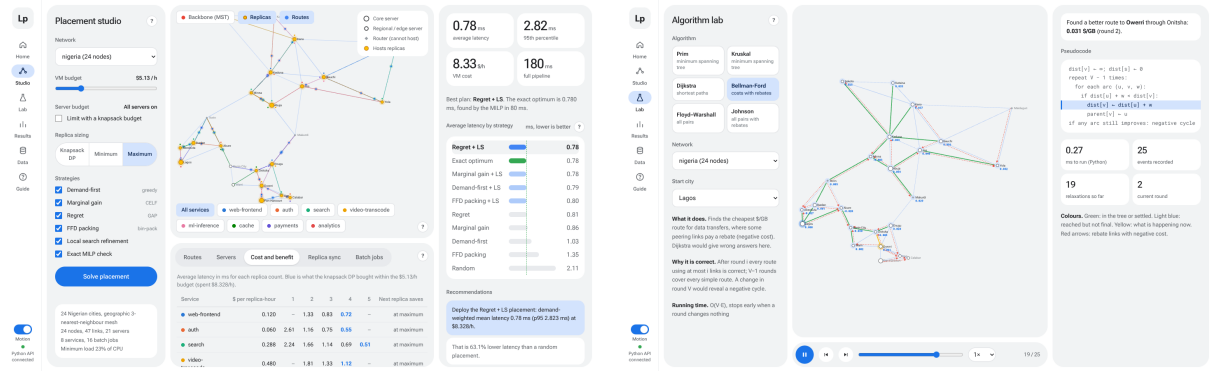
6 Implementation and Verification

Lightpath is a Python package (`cloudplace`) with modules for core data structures, algorithms, the problem model, data generators and the end-to-end solver. A FastAPI service exposes datasets, network generation, solving and step-by-step algorithm traces. A React/TypeScript dashboard (Fig. 7) offers a placement studio with live solving, an algorithm lab that replays traces alongside pseudocode, experiment charts, dataset downloads (JSON, Gephi GEXF, CSV) and a guided tour. When the Python service is not running, the dashboard falls back to saved results so it also works on a static host.

Testing. 191 automated tests check every algorithm against independent references. MST weights, single-source distances (including negative arcs) and all-pairs matrices from every heap and algorithm are compared with NetworkX [24] on random graphs. 0/1 and multiple-choice knapsack results are compared with brute force. The MILP is compared with exhaustive search on small placements, and it lower-bounds every heuristic. Further tests check heap operations with property-based testing, verify that greedy placements never exceed capacity, check that batch-schedule lanes never overlap, confirm that the transfer-cost graph never contains a negative cycle on any dataset (Lemma 1), and check that the web server cannot be tricked into serving files outside its directory. Table 5 breaks the suite down by file.

Table 5: Automated test suite (191 tests, all passing).

File	Tests	What is checked
`test_core.py`	15	Heap operations (property-based), union-find, graph construction
`test_paths.py`	66	MST, Dijkstra, Bellman-Ford, Floyd–Warshall and Johnson against NetworkX; negative-cycle detection; no negative cycle in any transfer graph
`test_optimisation.py`	100	Knapsack and MCKP against brute force; MILP against exhaustive search; greedies respect capacity; scheduler lanes never overlap; full pipeline on every dataset
`test_api.py`	10	Every endpoint, error codes, path-traversal guard



(a) Placement studio (Nigeria). **(b)** Algorithm lab: Bellman-Ford with negative arcs.

Figure 7: The Lightpath dashboard.

7 Experiments and Results

All timings are medians of repeated runs in CPython 3.13 on a single core. Quality experiments average three seeds. The full suite runs in about 80 seconds and writes CSV files and the figures below.

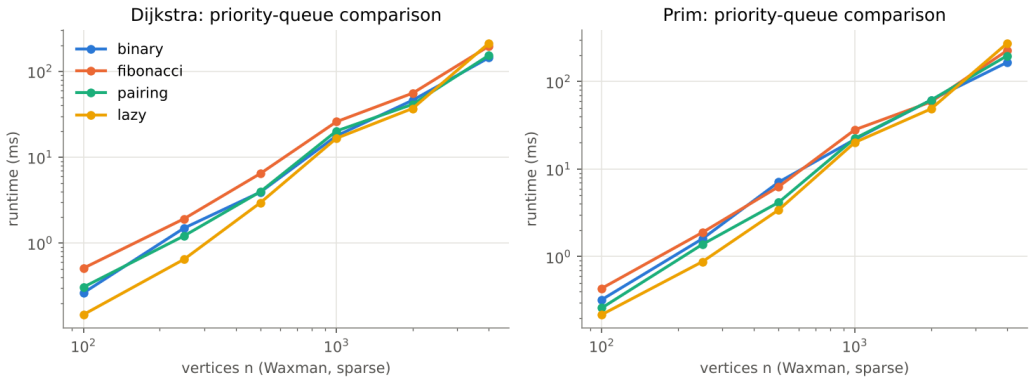


Figure 8: E1: Dijkstra and Prim with four priority queues on sparse Waxman graphs (n up to 4000, m up to 252k).

E1, priority queues (Fig. 8). The Fibonacci heap has the best asymptotic bound but is consistently slower: at $n = 1000$ Dijkstra takes 25.9 ms with it against 17.7 ms with the binary heap, and at $n = 4000$ 196 ms against 144 ms. Its pointer-based nodes and consolidation cost more in an interpreter than the saved $O(\log n)$ decrease-keys. `heapq` with lazy deletion (implemented in C) is fastest up to $n = 2000$ but falls behind at $n = 4000$ (212 ms) as stale entries accumulate on dense graphs. Bellman-Ford on the transfer graph costs 490 ms at $n = 4000$, reflecting its $O(nm)$ bound.

E2, all pairs (Fig. 9). On dense graphs the choice is clear-cut: at $n = 400$ vectorised Floyd–Warshall needs 277 ms against 5015 ms for Johnson and 5323 ms for repeated Dijkstra, and the rule correctly selects it. On sparse graphs repeated Dijkstra (724 ms) beats Johnson (985 ms), but the NumPy Floyd–Warshall is faster still (285 ms). The asymptotic rule is right about Dijkstra versus the pure-Python $\Theta(n^3)$ loop

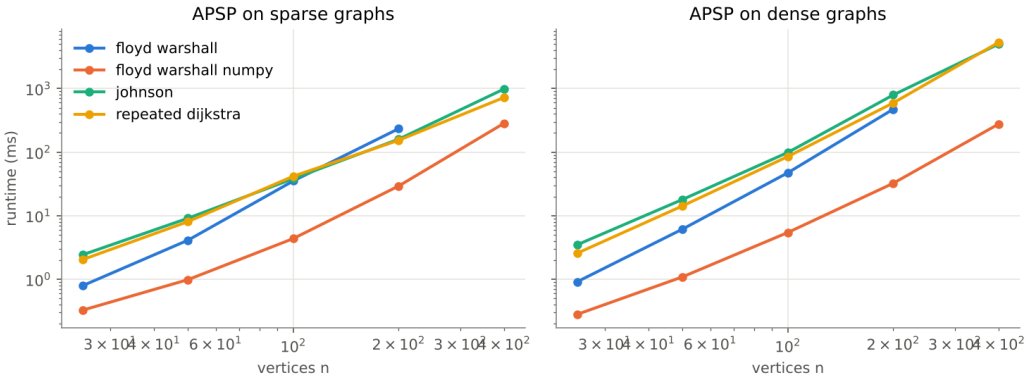


Figure 9: E2: all-pairs algorithms on sparse ($p = 3/n$) and dense ($p = 0.5$) graphs.

(234 ms already at $n = 200$), but vectorisation shifts the crossover well beyond $n = 400$. *Recommendation:* calibrate the threshold with the constant factors of the actual implementation rather than the asymptotics alone.

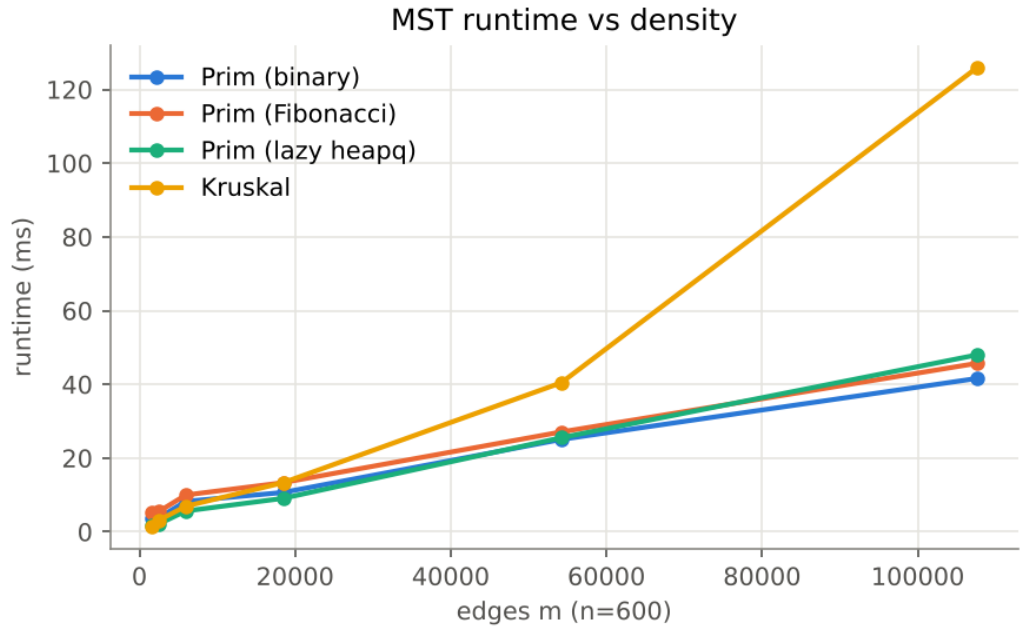


Figure 10: E3: Prim (four heaps) versus Kruskal as edge density grows ($n = 600$).

E3, MST versus density (Fig. 10) ($n = 600$, m from 1.5k to 108k). Kruskal is fastest on sparse graphs (1.5 ms against 3.7 ms for binary-heap Prim) and slowest on dense ones (126 ms against 42 ms), matching its $O(m \log m)$ sort against Prim's single pass per edge. All variants return identical tree weights, as the cut property requires.

E4, quality and cost versus size (Fig. 11). Averaged over $n \in \{20, \ldots, 240\}$, greedy alone is 8.7 % (marginal gain), 9.2 % (regret), 16.7 % (demand-first) and 38.4 % (FFD) above the reference, while a random placement is 167 % above. Local search reduces every method to 0.45–0.53 % on average and at most 3.0 %. The exact MILP grows from 27 ms at $n = 20$ to 1.7 s at $n = 160$, against 4 ms and 344 ms for regret plus local search: about five times cheaper, and the gap in growth rate widens with size. Greedy alone is one to two orders of magnitude cheaper than the MILP. Beyond roughly 60 000 MILP variables we skip the exact solve and report the LP bound. Table 6 lists the per-size gaps.

E5, capacity pressure (Fig. 13). At nominal capacity every method is feasible. At 0.35× capacity (34 % minimum load, more after sizing) demand-first, marginal gain and regret all fail to place every replica in every run, because latency-first choices fragment server capacity. FFD packing stays feasible at

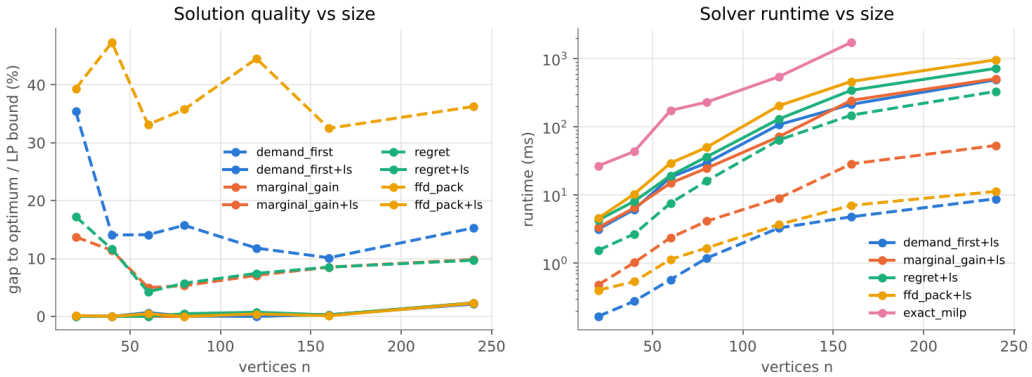


Figure 11: E4: optimality gap (left) and runtime (right) versus size on Waxman networks; dashed lines are greedy alone, solid lines include local search. At $n = 240$ the reference is the LP bound.

Table 6: E4: mean optimality gap (%) against the MILP optimum on Waxman networks (three seeds per size).

Strategy	$n = 20$	40	60	80	120	160
Random	186.63	163.01	165.61	137.93	166.80	172.12
FFD pack	39.35	47.34	33.14	35.75	44.54	32.50
Demand-first	35.41	14.06	14.11	15.73	11.80	10.12
Regret	17.23	11.61	4.24	5.73	7.44	8.51
Marginal gain	13.73	11.39	4.97	5.30	7.09	8.51
FFD pack + LS	0.14	0.00	0.44	0.00	0.41	0.11
Demand-first + LS	0.00	0.00	0.65	0.05	0.00	0.29
Regret + LS	0.00	0.00	0.00	0.45	0.72	0.24
Marginal gain + LS	0.14	0.00	0.00	0.00	0.52	0.24

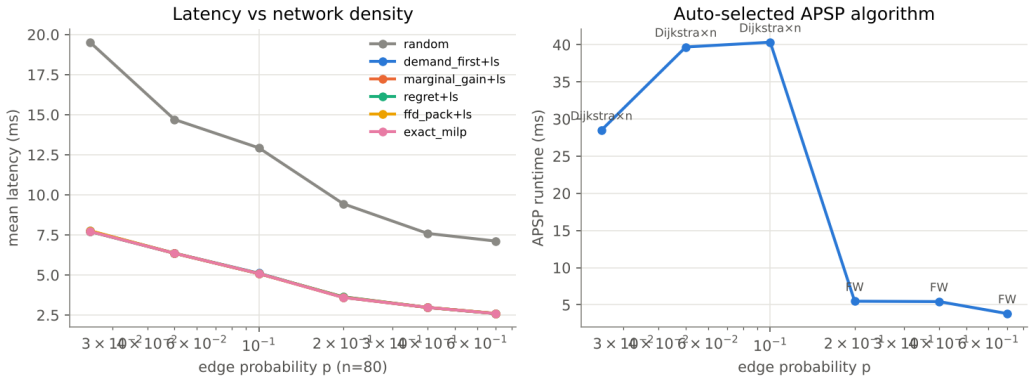


Figure 12: E6: optimal mean latency and heuristic gap versus edge probability (Erdős–Rényi, $n = 80$).

every capacity level, and local search then brings its latency within a few percent of optimal. Under tight capacity, feasibility and latency pull in different directions, and packing first is the safer strategy.

E6, density (Fig. 12). On Erdős–Rényi graphs with $n = 80$, raising the edge probability from 0.025 to 0.7 cuts the optimal mean latency from 7.71 ms to 2.59 ms, with diminishing returns beyond $p \approx 0.2$. Regret plus local search matches the optimum at five of six densities (0.9 % gap at $p = 0.2$). The automatic rule switches from repeated Dijkstra to Floyd–Warshall at $p = 0.2$.

E7, budget frontier. On the India network, raising the VM budget lets the multiple-choice knapsack buy replicas where they help most. Spending rises from \$2.41/h (9 replicas) to \$7.51/h (27 replicas) and mean latency falls from 4.34 ms to 1.28 ms (70 %), with diminishing returns. Beyond \$7.51/h every service is already at its maximum replica count. The minimum-replica plan stays at 4.34 ms regardless of budget.

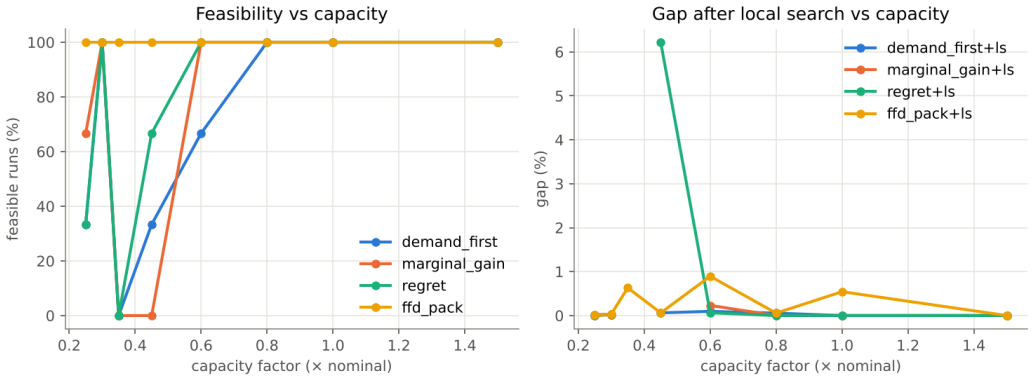
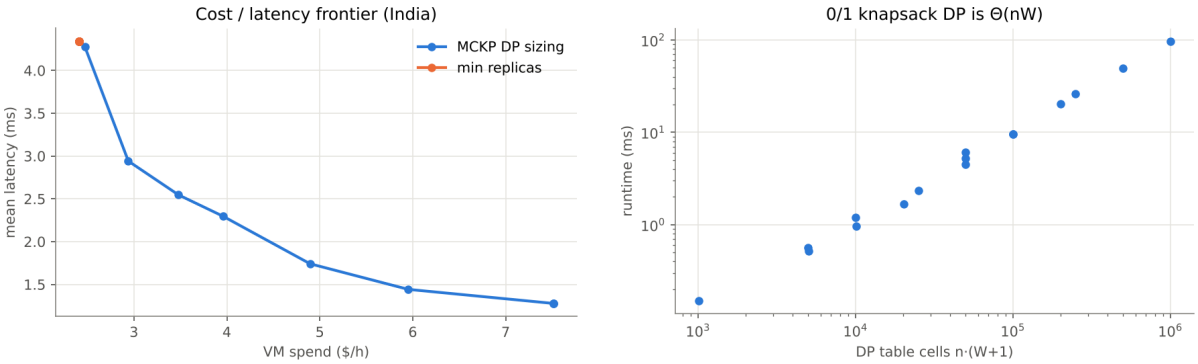


Figure 13: E5: share of runs with a feasible placement (left) and gap after local search (right) as server capacity shrinks (Waxman, $n = 50$, 12 services).

E8, knapsack scaling. Runtime grows linearly in the number of table cells $n(W + 1)$, from 0.15 ms (1 010 cells) to 96.9 ms (1 000 200 cells), roughly 97 ns per cell, which confirms the $\Theta(nW)$ bound. Fig. 14 shows E7 and E8 and Table 7 summarises all eight experiments.



(a) E7: VM budget versus latency (India).

(b) E8: knapsack runtime versus table size.

Figure 14: Budget frontier and knapsack scaling.

Table 7: Summary of the eight experiments.

Exp.	Varied	Main finding
E1	graph size, heap type	Binary heap fastest at scale; Fibonacci slower in practice despite better bound
E2	size, density	NumPy Floyd–Warshall wins on dense graphs; repeated Dijkstra beats Johnson on sparse
E3	density	Kruskal best when sparse, Prim best when dense; identical tree weight
E4	size	Greedy + LS within ≈0.5 % of optimum, about 5× faster than MILP
E5	capacity	Under tight capacity only FFD packing stays feasible
E6	density	More links cut latency 67 %, with diminishing returns after $p \approx 0.2$
E7	VM budget	\$2.41→\$7.51/h cuts latency 70 % (4.34→1.28 ms)
E8	DP table size	Linear in $n(W + 1)$, about 97 ns per cell

8 Case Studies and Policy Outputs

For every instance the solver emits a route table (client → replica, path, latency, backbone-only stretch), a server schedule (VMs and utilisation), a cost/benefit table (latency for every replica count and the saving per extra dollar), cheapest replica-sync routes, and a batch Gantt chart. Table 8 and Figs. 15–16 summarise the three main networks.

India. The 24-link MST backbone totals 56.2 ms. The best plan matches the MILP optimum. Mumbai, Pune, Kochi, Chandigarh and Varanasi run at ≥ 90 % CPU, so capacity there is the binding constraint,

Table 8: Recommended plans. Gap is the best heuristic's distance from the MILP optimum; stretch is the latency increase if requests were routed over the MST backbone only.

Network	Mean (ms)	p95 (ms)	vs. random	Best heuristic	MILP time	Stretch	Best next replica
India	2.326	10.16	−64.0%	0.00 % in 4.1 ms	27 ms	1.55×	video-transcode: −2.45 ms for $0.48
Nigeria	1.137	2.88	−44.3%	0.09 % in 9.6 ms	70 ms	1.23×	analytics: −1.23 ms for $0.58/h
Woxsen	0.055	0.21	−38.9%	0.00 % in 2.9 ms	20 ms	1.39×	dining POS: −0.028 ms for $0.06/h

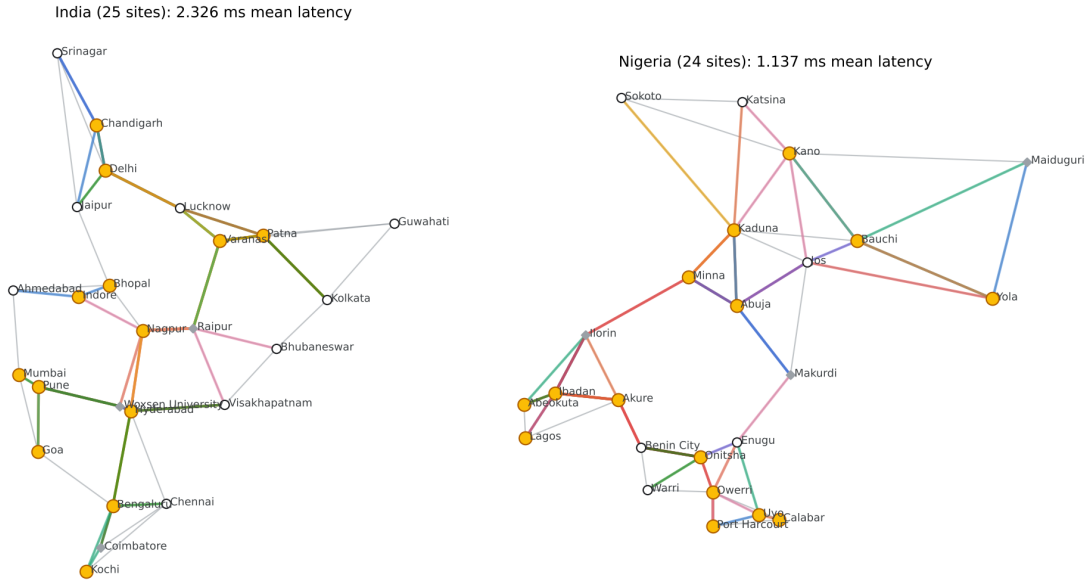


Figure 15: Recommended placements for India and Nigeria. Yellow sites host replicas; coloured lines are request routes (one colour per service); grey diamonds are routers.

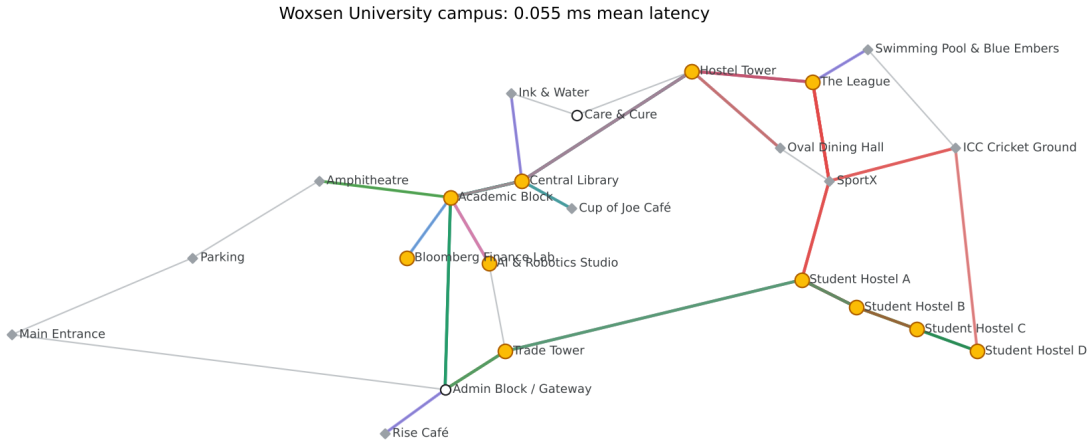


Figure 16: Woxsen University campus plan. Replicas cluster around the Academic Block, the hostels and The League sports arena, where demand originates.

and upgrading those servers is the highest-value hardware change. **Nigeria.** The exact MILP is best (1.137 ms); marginal gain plus local search is within 0.09 %. Lagos, Abuja and Port Harcourt host the core VMs, while Onitsha and Owerri saturate. **Woxsen.** Everything is within 0.52 ms. Hostel and sports traffic dominate, so the learning portal and Wi-Fi authentication replicate into Student Hostels A and D, and live streaming runs at The League.

9 Recommendations

1. **Default solver:** run all four greedy schedulers with local search and keep the best. This lands within 0.5 % of optimal on average at a fraction of the exact cost. Run the MILP as a certificate whenever it has fewer than about 60k variables.
2. **Capacity first when load is high:** when minimum-replica load exceeds about 30 %, start from FFD packing, then refine for latency (E5).
3. **Spend by marginal value:** size replicas with the multiple-choice knapsack and buy the next replica with the best ms-per-dollar ratio from the cost/benefit table. Returns diminish quickly (E7).
4. **Provision the MST backbone first, but route on the full graph:** routing over the tree alone would stretch latency by 1.2–1.6×.
5. **Engineering in Python:** prefer binary heaps or `heapq` over Fibonacci heaps (E1), and choose all-pairs algorithms with constants measured on the actual implementation (E2).
6. **Transfer pricing:** with negative costs, use Bellman-Ford or Johnson only. The potential form (3) keeps rebates safe from negative cycles.

10 Limitations and Future Work

India and Nigeria use real city locations but synthetic links, and Woxsen's fibre plan and server sizes are modelling assumptions. Demand is static, and latency ignores queueing and congestion. The activation stage uses an additive surrogate value, so it is not optimal for the joint problem. Timings reflect CPython constants. Natural extensions are online re-placement as demand shifts, queueing-aware latency (for example M/M/c servers), multi-objective cost–latency fronts, carrier topology data such as the Internet Topology Zoo [25], and a compiled core for larger instances.

11 Conclusion

Lightpath shows how classical algorithm design fits a practical systems problem. Exact algorithms (MST, Dijkstra, Bellman-Ford, Floyd–Warshall, Johnson) and pseudo-polynomial dynamic programs handle the tractable parts. Greedy schedulers with local search handle the NP-hard placement. An exact MILP shows the heuristics are within 0.5 % of optimal on average, while the experiments expose where asymptotic theory and practical constants disagree. The same pipeline produces deployable policies for national networks in India and Nigeria and for the Woxsen campus.

References

[1] O. Kariv and S. L. Hakimi, "An algorithmic approach to network location problems. II: The p-medians," *SIAM Journal on Applied Mathematics*, vol. 37, no. 3, pp. 539–560, 1979.

[2] B. M. Waxman, "Routing of multipoint connections," *IEEE Journal on Selected Areas in Communications*, vol. 6, no. 9, pp. 1617–1622, Dec. 1988.

[3] P. Erdős and A. Rényi, "On random graphs I," *Publicationes Mathematicae Debrecen*, vol. 6, pp. 290–297, 1959.

[4] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999.

[5] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," in *Proc. ACM SIGCOMM*, Seattle, WA, USA, 2008, pp. 63–74.

[6] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, Jul. 1987.

[7] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, Apr. 1975.

[8] R. C. Prim, "Shortest connection networks and some generalizations," *Bell System Technical Journal*, vol. 36, no. 6, pp. 1389–1401, Nov. 1957.

[9] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, 1956.

[10] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[11] R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

[12] L. R. Ford, Jr., "Network flow theory," RAND Corporation, Santa Monica, CA, USA, Paper P-923, 1956.

[13] R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, vol. 5, no. 6, p. 345, Jun. 1962.

[14] S. Warshall, "A theorem on Boolean matrices," *Journal of the ACM*, vol. 9, no. 1, pp. 11–12, Jan. 1962.

[15] D. B. Johnson, "Efficient algorithms for shortest paths in sparse networks," *Journal of the ACM*, vol. 24, no. 1, pp. 1–13, Jan. 1977.

[16] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.

[17] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance, "Cost-effective outbreak detection in networks," in *Proc. 13th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Jose, CA, USA, 2007, pp. 420–429.

[18] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. Chichester, U.K.: Wiley, 1990.

[19] G. Dósa, "The tight bound of first fit decreasing bin-packing algorithm is $\mathrm{FFD}(I) \leq 11/9\,\mathrm{OPT}(I) + 6/9$," in *Combinatorics, Algorithms, Probabilistic and Experimental Methodologies (ESCAPE 2007)*, LNCS vol. 4614. Berlin, Germany: Springer, 2007, pp. 1–11.

[20] V. Arya, N. Garg, R. Khandekar, A. Meyerson, K. Munagala, and V. Pandit, "Local search heuristics for k-median and facility location problems," *SIAM Journal on Computing*, vol. 33, no. 3, pp. 544–562, 2004.

[21] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[22] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[23] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.

[24] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy 2008)*, Pasadena, CA, USA, 2008, pp. 11–15.

[25] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The Internet Topology Zoo," *IEEE Journal on Selected Areas in Communications*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.